Práctica 3.3 — Contenerizar y orquestar el servidor de Nebula Arena

Objetivo: construir la imagen de un servidor de partida en WebSocket y desplegarlo como flota de Agones.

Entregado en AWS (EKS, 2026-11-09)

Pieza	Cómo quedó
Registro	ECR `nebula-server` y `matchmaker` (`infra-aws/registro.tf`)
Red	VPC con 2 subredes **públicas** y sin NAT Gateway (ahorro de ~USD 33 al mes). Puertos 7000–8000 TCP/UDP abiertos para las partidas (`infra-aws/red.tf`)
Clúster	EKS **1.36** (soporte estándar, compatible con Agones 1.62). Nodos `m7i-flex.large` (2 vCPU, 8 GB, permitidos por el plan gratuito), mínimo 2 y máximo 6 (`infra-aws/cluster.tf`)
Agones	1.62.0 con Helm desde Terraform, sin balanceadores innecesarios (`infra-aws/complementos.tf`)
Flota	`k8s/aws/fleet.yaml`: 3 réplicas; cada partida manda su resultado a la tabla de líderes (3.5) con la llave en un Secret

Resultados: - La flota pasó a **3 servidores** `Ready` **en 8 s** después de `kubectl apply`. - **Partida real por internet:** `POST /match` al matchmaker → servidor `nebula-fleet-…-2nt49` en `ec2-3-139-56-74…:7981`. Jugaron **10 jugadores durante 15 s** desde la PC: 130 pings, **RTT p50 = 106 ms**, p95 = 133 ms, p99 = 147 ms. Es la distancia de Colima a Ohio, la única región que permite la cuenta. - **Ciclo completo de Agones:** el servidor terminó su partida → `Shutdown` → la flota creó uno nuevo para mantener 3 `Ready`. - **Evidencia:** `entrega/evidencias/3.3-cluster-agones/`.

Todo se crea con `terraform apply` y se borra con `terraform destroy` (ver `infra-aws/README.md`).

Versión original para Google Cloud y ensayo local

Paso	Estado
1. Servidor (`server/server.py`)	Hecho y probado
2. Dockerfile, imagen y `/metrics`	Hecho y probado en local
3. Artifact Registry, clúster GKE y Agones	Ensayado en local con kind + Agones 1.62. **En GKE: pendiente, necesita la cuenta de Google Cloud**
4. Fleet y GameServerAllocation	Funcionando en kind (evidencia abajo). **En GKE: pendiente**

El ensayo en kind demuestra que la imagen, el SDK de Agones y la flota funcionan. **No sustituye** las capturas del entregable, que deben salir de GKE: kind corre en una sola máquina, sin Cluster Autoscaler ni IP pública.

Paso 1 — Servidor

`server/server.py` parte del servidor mínimo del material y conserva su protocolo y los nombres de sus métricas (las consultas de Grafana de la 3.4 dependen de ellos):

- Hasta 10 jugadores (5 vs 5). El 11.º recibe un cierre con código `4000` ("partida llena").
- Retransmite el estado a 20 ticks/s y descuenta lo que tarda cada tick para mantener 20 Hz reales.
- `{"ping": t}` se responde al instante con `{"pong": t}`, para medir RTT.
- Ciclo de vida de Agones por REST: `/ready` al arrancar, `/health` cada 2 s y `/shutdown` cuando se van todos los jugadores.

Agregados sobre el ejemplo del material:

- **Validación de entradas:** se descartan el JSON roto, los tipos incorrectos, los `NaN`, las coordenadas fuera de la arena de 100 × 100 y los ids vacíos o de más de 32 caracteres. Un mensaje inválido no tumba la conexión.
- **Anti-suplantación:** cada conexión queda ligada al primer id que envía y no puede escribir la posición de otro jugador.
- **Métricas extra:** `nebula_tick_rate` (ticks/s efectivos), `nebula_disconnects_total{motivo}` y `nebula_invalid_messages_total`.

Pruebas

```
cd server
../.venv/Scripts/python -m pytest -q        # en Linux/Mac: ../.venv/bin/python
```

13 pruebas: 9 unitarias (`tests/test_server.py`, incluidas las 2 del material) y 4 de funcionamiento (`tests/test_integracion.py`). Las de funcionamiento levantan el servidor real y conectan clientes WebSocket: ping/pong, snapshot, mensajes inválidos, rechazo del 11.º jugador y limpieza del estado al salir. Resultado: **13 passed**.

Paso 2 — Imagen Docker

```
cd server
docker build -t nebula-server:1.2 .
docker run -p 7654:7654 -p 9100:9100 nebula-server:1.2
curl localhost:9100/metrics | grep nebula
```

Imagen `python:3.11-slim`, usuario sin privilegios (`USER 1000`), 206 MB. La versión actual es `1.2`: la 1.1 agregó el manejo de `SIGTERM` (ver abajo) y la 1.2 reintenta `/ready` hasta que el sidecar de Agones responde (hallazgo de la 3.4). El `.dockerignore` deja fuera las pruebas y los entornos virtuales.

Prueba con 10 jugadores

`server/cliente_prueba.py` conecta N jugadores que envían su posición 20 veces por segundo y hacen un ping por segundo:

```
python cliente_prueba.py --url ws://localhost:7654 --jugadores 10 --segundos 10
```

Medición del 2026-11-05 en la laptop (Windows 11, Docker Desktop):

Escenario	Pings	RTT p50	RTT p95	RTT p99
Contenedor Docker, 10 jugadores	90	5.82 ms	74.68 ms	86.71 ms
Servidor nativo (sin Docker), 10 jugadores	80	2.16 ms	4.30 ms	4.69 ms
Servidor nativo, 1 jugador	8	0.59 ms	0.81 ms	0.81 ms

Con el contenedor, `/metrics` reportó `nebula_tick_rate` = 19.69 (98 % del objetivo de 20), `nebula_disconnects_total{motivo="normal"}` = 10 y 0 mensajes inválidos. El log registró "partida terminada: todos los jugadores salieron", que es el momento en que el servidor llama a `/shutdown`.

Interpretación: con el mismo código, el p95 baja de 75 ms a 4 ms fuera de Docker. La cola de latencia la agrega el reenvío de puertos de Docker Desktop en Windows (WSL2), no el servidor. En GKE no existe esa capa, así que la medición que vale para el reporte es la de la práctica 3.4, con Locust en una VM de la misma región.

Ensayo local: kind + Agones (pasos 3 y 4 sin nube)

Evidencia completa, con la salida real de cada comando: `docs/evidencias/3.3-agones-local.txt` (2026-11-05). Kubernetes 1.37.0 (kind 0.33.0) y Agones 1.62.0.

```
winget install Kubernetes.kind
kind create cluster --config k8s/local/kind-config.yaml        # clúster "nebula"
helm repo add agones https://agones.dev/chart/stable && helm repo update
helm install agones agones/agones -n agones-system --create-namespace --wait
docker build -t nebula-server:1.2 server
kind load docker-image nebula-server:1.2 --name nebula         # la imagen va al nodo
kubectl apply -f k8s/local/fleet-local.yaml                    # misma flota, imagen local
kubectl get gameservers
kubectl create -f k8s/allocation.yaml
kubectl port-forward pod/<gameserver> 17654:7654 19100:9100    # para conectar jugadores
python server/cliente_prueba.py --url ws://localhost:17654 --jugadores 10 --segundos 15
kind delete cluster --name nebula                              # al terminar
```

`k8s/local/fleet-local.yaml` es idéntico a `k8s/fleet.yaml`, salvo la imagen (`nebula-server:1.2` con `imagePullPolicy: IfNotPresent`).

Qué se comprobó:

Comprobación	Resultado
3 GameServers arrancan y llaman a `/ready`	Los 3 en `Ready` a los 20 s (evento `SDK.Ready() complete`)
La asignación toma uno	`Ready → Allocated`
10 jugadores en el servidor asignado	`nebula_players_connected 10`, `nebula_tick_rate 19.68`. RTT por port-forward: p50 9.96 ms, p95 21.75 ms, p99 26.94 ms (140 pings)
Los jugadores salen y el servidor llama a `/shutdown`	`Allocated → Shutdown`, Agones borra el pod y la flota crea un servidor nuevo en `Ready` (en menos de 7 s)
Actualizar la flota de 1.0 a 1.1	Se reemplazan los servidores libres; en ~50 s los 3 estaban en 1.1

El RTT de este ensayo pasa por `kubectl port-forward`, que agrega su propia latencia. No es una medición válida para el reporte (esa es la de la 3.4).

Paso 3 — Clúster GKE y Agones (pendiente)

Necesita la cuenta de Google Cloud. Los comandos son los del material:

```
gcloud services enable container.googleapis.com artifactregistry.googleapis.com
compute.googleapis.com
gcloud artifacts repositories create nebula --repository-format=docker --location=us-central1
gcloud auth configure-docker us-central1-docker.pkg.dev
IMG=us-central1-docker.pkg.dev/<PROYECTO>/nebula/nebula-server:1.2
docker build -t $IMG server && docker push $IMG

gcloud container clusters create nebula --zone us-central1-a \
  --machine-type e2-standard-4 --num-nodes 3 \
  --enable-autoscaling --min-nodes 3 --max-nodes 12 --tags game-server
gcloud compute firewall-rules create agones-gs --allow udp:7000-8000,tcp:7000-8000 \
  --target-tags game-server

helm repo add agones https://agones.dev/chart/stable && helm repo update
helm install agones agones/agones -n agones-system --create-namespace
```

Ojo con la prueba gratuita: 3 nodos `e2-standard-4` son **12 vCPU**, y la prueba gratuita limita a **8 vCPU** encendidas a la vez. Opciones: activar la cuenta completa (conserva los créditos), o usar `--num-nodes 2 --min-nodes 2` con `e2-standard-4` (8 vCPU) solo para esta práctica.

Al terminar la sesión, borra el clúster para que no siga cobrando:

```
gcloud container clusters delete nebula --zone us-central1-a
```

Paso 4 — Flota y asignación (pendiente de aplicar)

```
# en k8s/fleet.yaml reemplaza <PROYECTO> por tu proyecto
kubectl apply -f k8s/fleet.yaml
kubectl get gameservers               # deben aparecer 3 en estado Ready
kubectl create -f k8s/allocation.yaml
kubectl get gameservers               # uno pasa a Allocated
```

Entregable

- [x] Captura de `docker run` → `docker run -p 7654:7654 -p 9100:9100 nebula-server:1.2`
- [x] Captura de `/metrics` → `curl localhost:9100/metrics | grep nebula`
- [ ] Captura de `kubectl get gameservers` antes y después de la asignación en GKE (ensayo en kind ya hecho: `docs/evidencias/3.3-agones-local.txt`)
- [x] Explicación de qué pasa si se borra un pod `Allocated` (abajo)

¿Qué pasa si borras un pod en estado `Allocated`?

Lo probamos en el clúster local (pasos 7–10 de la evidencia). Un `GameServer` en `Allocated` tiene una partida en curso, y su estado (posiciones, puntuación) vive solo en la memoria de ese proceso. Al ejecutar `kubectl delete pod <allocated>`:

1. Kubernetes manda `SIGTERM` a los contenedores (evento `Killing`). Las conexiones de los jugadores se cortan y **la partida se pierde**, porque no hay otra copia de ese estado.
2. Mientras el pod se apaga, el `GameServer` sigue diciendo `Allocated`. Agones no lo da por perdido hasta que su sidecar deja de responder: se registran `Liveness probe failed` y `Unhealthy`, y luego `Deletion started`.
3. La `Fleet` ve que le falta una réplica y crea un `GameServer` **nuevo** en `Ready`. No es una recuperación: el servidor nuevo está vacío y la partida anterior no regresa.

Hallazgo del experimento: con la versión 1.0 el pod tardó **30 s** en morir. En el contenedor, Python es el proceso 1 y, sin un manejador, ignora `SIGTERM`; el pod esperó todo el periodo de gracia hasta recibir `SIGKILL`. La versión 1.1 maneja `SIGTERM` (cancela el tick y cierra las conexiones), y el mismo borrado tardó **2 s**. El reemplazo también llega antes.

Por eso Kubernetes nunca debe apagar un servidor `Allocated` por su cuenta. Agones evita que el escalado o una actualización de la flota lo eliminen (en el paso 9 la actualización solo reemplazó servidores libres) y espera a que el propio servidor llame a `/shutdown`. Un borrado manual se salta esa protección.